

Macro `mAABB_CHECK`

Vue d'ensemble

`mAABB_CHECK` est une macro assembleur Z80 qui teste le chevauchement entre deux rectangles alignés sur les axes (AABB — *Axis-Aligned Bounding Box*). Elle est destinée aux moteurs de jeu Amstrad CPC pour la détection de collision entre entités rectangulaires (sprites, raquettes, murs, zones de déclenchement, etc.).

Contrairement à une routine appelée via `CALL`, cette macro génère du code **inline** : son corps est recopié intégralement à chaque site d'utilisation, sans coût d'appel/retour (`CALL` / `RET`), au prix d'une taille de code plus importante par site d'appel.

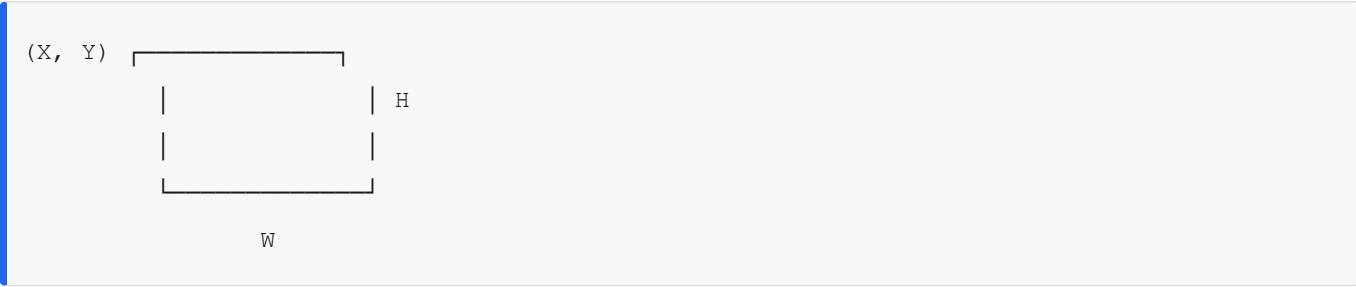
Signature

```
MACRO mAABB_CHECK
    ...
ENDM
```

Paramètres	Aucun
Entrée	Variables globales <code>AABB1_X</code> , <code>AABB1_W</code> , <code>AABB1_Y</code> , <code>AABB1_H</code> (rectangle 1) et <code>AABB2_X</code> , <code>AABB2_W</code> , <code>AABB2_Y</code> , <code>AABB2_H</code> (rectangle 2), supposées contiguës en mémoire
Sortie	Flag <code>Carry</code> : <code>1</code> si chevauchement, <code>0</code> sinon
Registres détruits	<code>AF</code> , <code>B</code> , <code>HL</code>

Convention géométrique

Chaque rectangle est décrit par son coin haut-gauche `(X, Y)` et ses dimensions `(W, H)` :



Principe algorithmique

Deux rectangles se chevauchent si et seulement si leurs projections sur **chacun** des deux axes (X et Y) se recouvrent. La macro effectue donc quatre tests de séparation, un par bord :

Test	Condition de séparation	Signification
1	<code>X2 ≥ X1 + W1</code>	Le rectangle 2 commence après la fin du rectangle 1 (séparation à droite)
2	<code>X1 ≥ X2 + W2</code>	Le rectangle 1 commence après la fin du rectangle 2 (séparation à gauche)
3	<code>Y2 ≥ Y1 + H1</code>	Le rectangle 2 commence après la fin du rectangle 1 (séparation en bas)
4	<code>Y1 ≥ Y2 + H2</code>	Le rectangle 1 commence après la fin du rectangle 2 (séparation en haut)

Si **l'un quelconque** de ces quatre tests est vrai, les rectangles sont nécessairement disjoints : la macro sort immédiatement avec `Carry = 0`. Ce n'est que si **aucun** des quatre tests ne se vérifie que le chevauchement est confirmé.

Ce court-circuit (sortie dès le premier test positif) évite de calculer inutilement les tests suivants dans le cas — le plus fréquent en pratique — où les objets sont éloignés.

Déroulé du code, étape par étape

Test X (1) — séparation à droite

```
LD    HL, AABB1_X
LD    A, (HL)           ; A = X1
INC   HL
ADD   A, (HL)           ; A = X1 + W1
LD    B, A              ; B = fin1_x
LD    HL, AABB2_X
LD    A, (HL)           ; A = X2
CP    B
JR    NC, @no_collision ; X2 >= fin1_x -> séparés
```

Calcule le bord droit du rectangle 1 (`X1 + W1`), puis compare la position gauche du rectangle 2. Exploite la contiguïté mémoire de `AABB1_X`/`AABB1_W` : un simple `INC HL` suffit à passer de l'un à l'autre, sans recharger `HL`.

Test X (2) — séparation à gauche

Symétrique du précédent, en inversant les rôles des deux rectangles.

Test Y (1) et Y (2)

Même logique, appliquée aux champs `AABB1_Y`/`AABB1_H` et `AABB2_Y`/`AABB2_H`.

Conclusion

```
SCF                                ; aucun test de séparation n'a été vrai -> chevauchement
JR    @end_check
@no_collision:
OR    A                            ; Carry = 0
@end_check:
```

Sémantique du contact bord à bord

Chaque test utilise une comparaison **large** ($\geq$, via `CP` + `JR NC`) pour déclarer la séparation. Concrètement, si `x2` est exactement égal à `x1 + w1` (les deux rectangles se touchent sans se recouvrir), le test 1 se déclenche et la macro renvoie `Carry = 0` : **le contact exact n'est pas considéré comme une collision.**

 Ce point mérite une attention particulière selon l'usage prévu. Pour une détection de collision balle/raquette où le premier instant de contact doit compter, ce comportement peut retarder la détection d'une frame par rapport à une variante en comparaison stricte inclusive. À l'inverse, pour des tests de zone ou de déclenchement où le contact exact ne doit pas suffire, ce comportement est le plus approprié.

Utilisation de `@no_collision` / `@end_check`

Les étiquettes préfixées par `@` sont des labels **locaux à l'expansion de la macro** (portée Rasm). Cette convention est indispensable : comme le corps de la macro est dupliqué à chaque site d'appel, des labels globaux classiques provoqueraient une erreur d'assemblage (« label déjà défini ») dès le second appel dans le même fichier.

Exemple d'utilisation

```
; Remplir les boîtes de collision

LD  A, (ENT_BALL.PosX)
LD  (AABB1_X), A
LD  A, BALL_PX_WIDTH_M0
LD  (AABB1_W), A
LD  A, (ENT_BALL.PosY)
LD  (AABB1_Y), A
LD  A, BALL_PX_HEIGHT_M0
LD  (AABB1_H), A

LD  A, (ENT_PADDLE.PosX)
LD  (AABB2_X), A
LD  A, PADDLE_PX_WIDTH_M0
LD  (AABB2_W), A
LD  A, (ENT_PADDLE.PosY)
LD  (AABB2_Y), A
LD  A, PADDLE_PX_HEIGHT_M0
LD  (AABB2_H), A

mAABB_CHECK
JR   NC, .noCollision
; ... traitement de la collision ...
.noCollision
```

Caractéristiques de performance

Aspect	Détail
Taille par site d'appel	~46 octets (dépliée intégralement, aucune factorisation)
Coût en cycles (meilleur cas)	Séparation détectée dès le 1 ^{er} test : ~40 T-states
Coût en cycles (pire cas)	Chevauchement confirmé, les 4 tests exécutés : ~160 T-states
Absence de  /  	Économise ~27 T-states par appel par rapport à une routine équivalente, au prix de la duplication du code à chaque site

Comparaison avec une routine `CALL`-able équivalente

	Macro inline (<code>mAABB_CHECK</code>)	Routine <code>CALL</code> (<code>F_AABB_CHECK</code>)
Vitesse	Plus rapide (pas de <code>CALL</code> / <code>RET</code>)	Légèrement plus lente
Taille totale	Croît avec le nombre de sites d'appel	Fixe, une seule copie en mémoire
Registres détruits	<code>AF</code> , <code>B</code> , <code>HL</code>	Selon l'implémentation (potentiellement plus, ex. <code>DE</code> si factorisée par axe)
Cas d'usage recommandé	Un ou deux sites d'appel, section critique en performance	Plusieurs sites d'appel, contrainte mémoire

Recommandations d'intégration

- **Vérifier la contiguïté mémoire** de `AABB1_X` / `AABB1_W` et `AABB2_X` / `AABB2_W` (de même pour `Y` / `H`) avant d'utiliser cette macro : le `INC HL` suivi de `(HL)` suppose que `W` suit immédiatement `X` en mémoire, sans octet intercalaire.
- **Ne pas confondre** cette macro avec une variante à seuil large (contact inclus) : si le comportement souhaité diffère selon le contexte d'appel (ex. détection de but vs détection de rebond), prévoir une seconde macro ou un paramètre conditionnel plutôt que de modifier celle-ci au risque de casser un autre appelant.
- **Documenter le contrat de destruction de registres** (`AF` , `B` , `HL`) dans les routines appelantes, en particulier si elles supposent la préservation d'un de ces registres à travers l'appel — un contrat de destruction non respecté par l'appelant est une source de bug classique et difficile à diagnostiquer sur ce type de code.